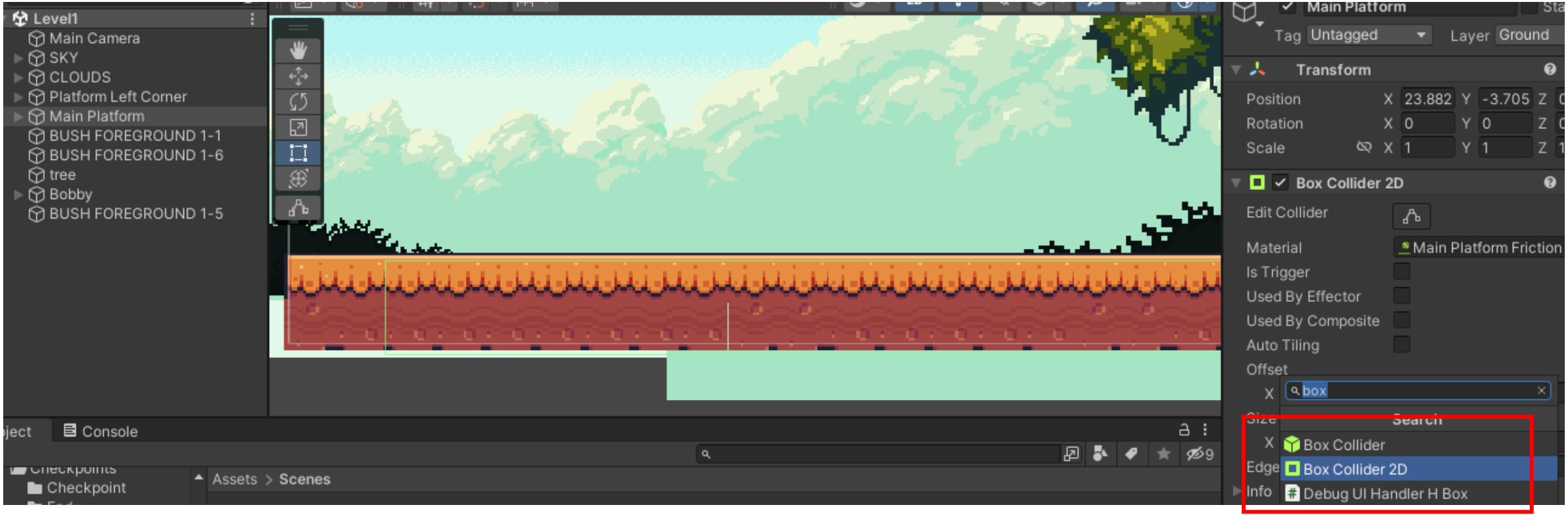


Player creation, and basic movement

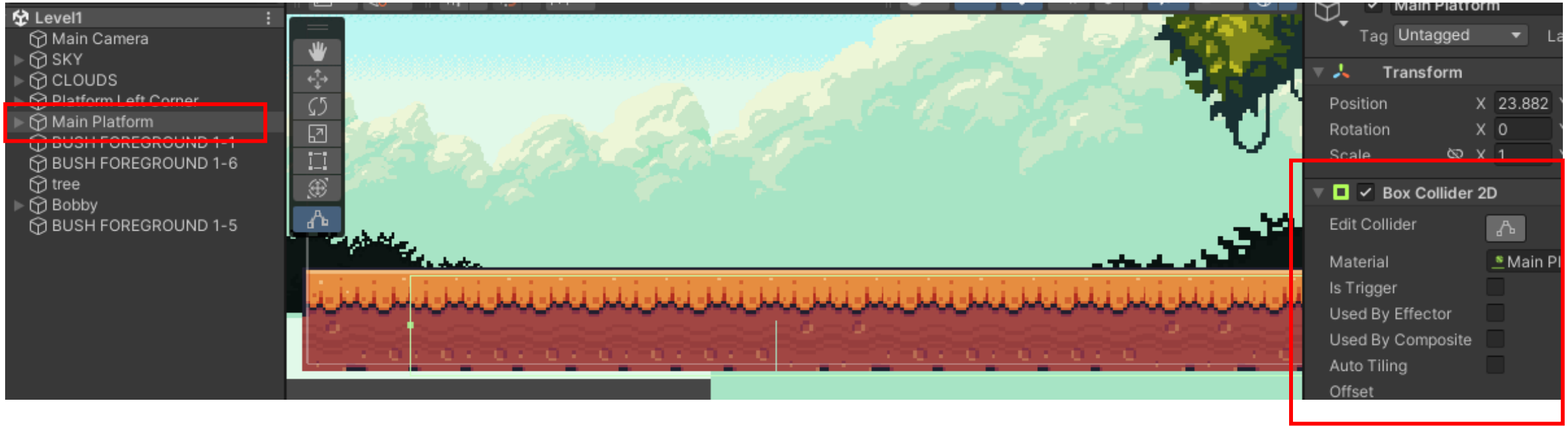
Exercise #02 – Make the Ground Solid

- You'll need to make the ground solid for your character to stand on.
- Select ground from the Hierarchy > Go to Inspector > Click Add Component > Physics2D > BoxCollider2D



Exercise #02 – Make the Ground Solid (Cont.)

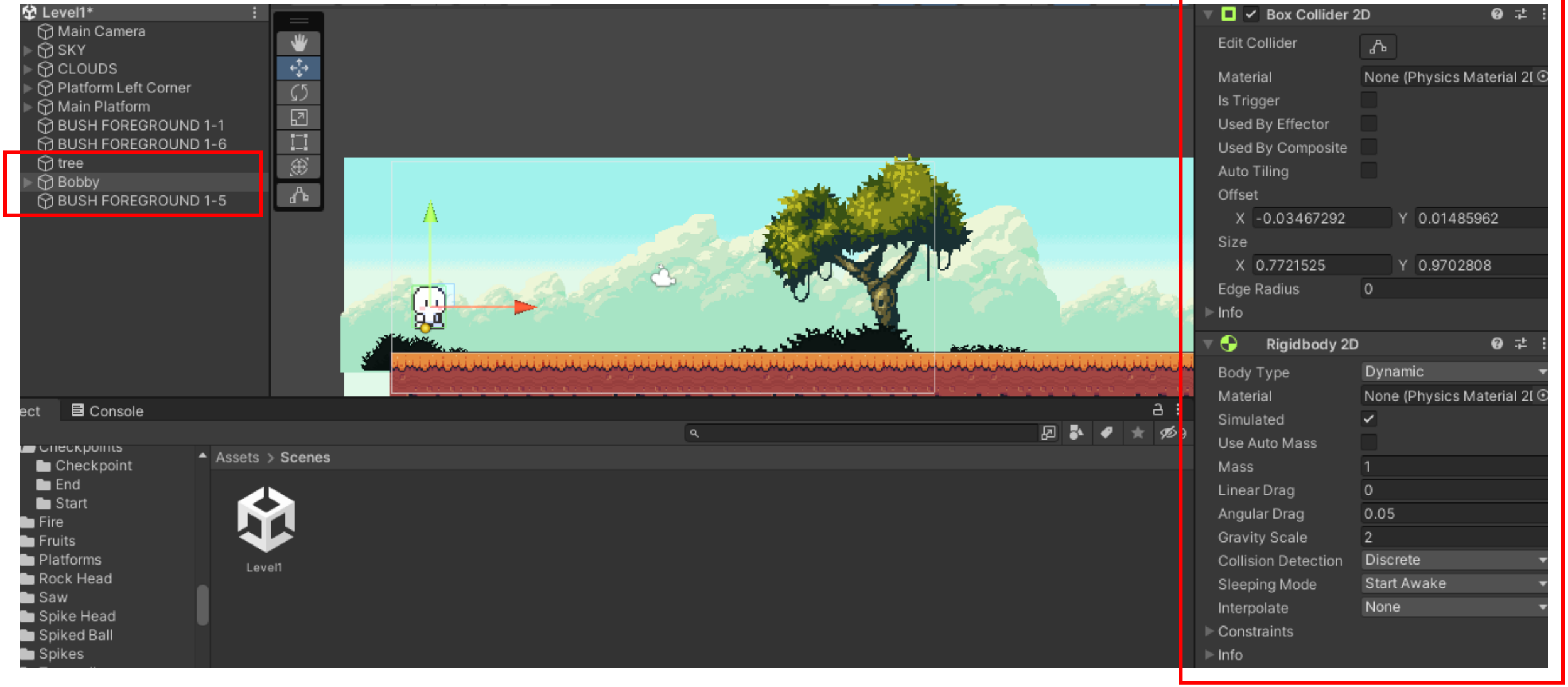
- To adjust the size of the (green) collider around the ground, Click on Edit Collider in the BoxCollider2D Component and adjust it as you need.
- Now the ground is solid.



Exercise #02 – Make the Ground Solid (Cont.)

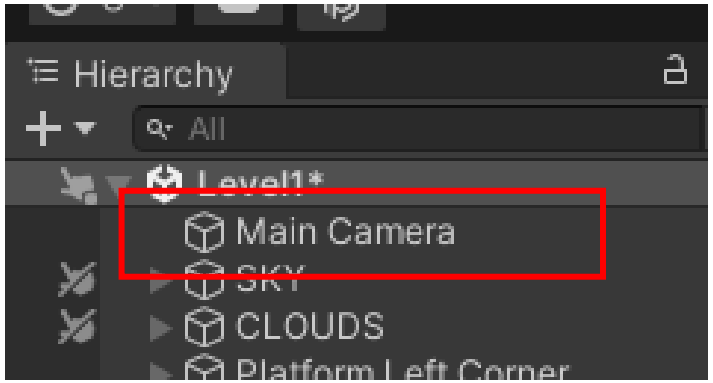
- Using exercise #02, add a suitable 2D collider to your player character.
- Click the Play button, but still nothing happens!
- To make the player character respond to physics, a *RigidBody2D* must be added
- Select the player from the Hierarchy > click Add Component > Physics2D >RigidBody2D
- As you'll see, the Rigidbody component has many useful settings. Experiment with Gravity. Suspend the character up in the sky, then click the Play button to see them fall and hit the ground
- Click the play button again to stop Game Mode and change the gravity's value to 10. Click play again. What happens?

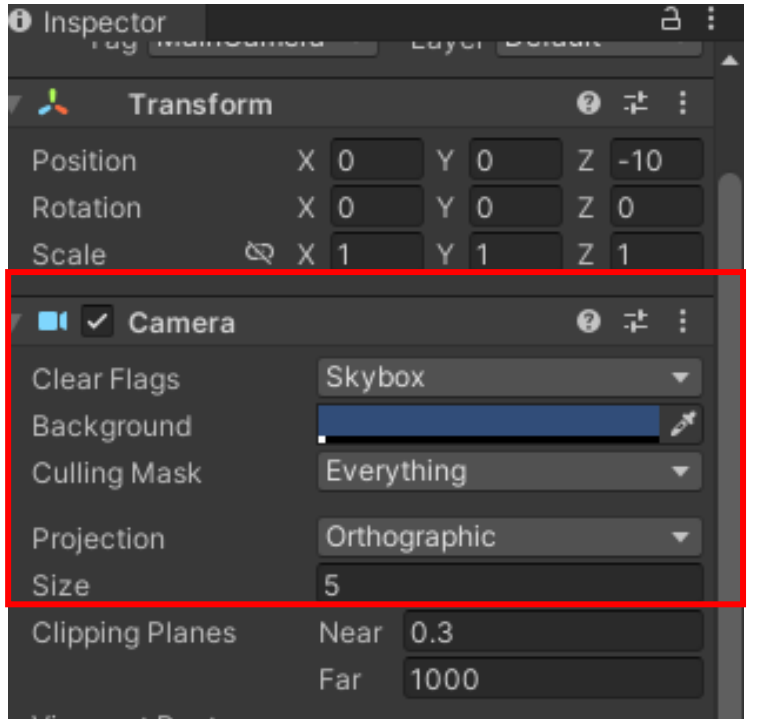
Exercise #03 – Make the Character Solid (Cont.)



Camera

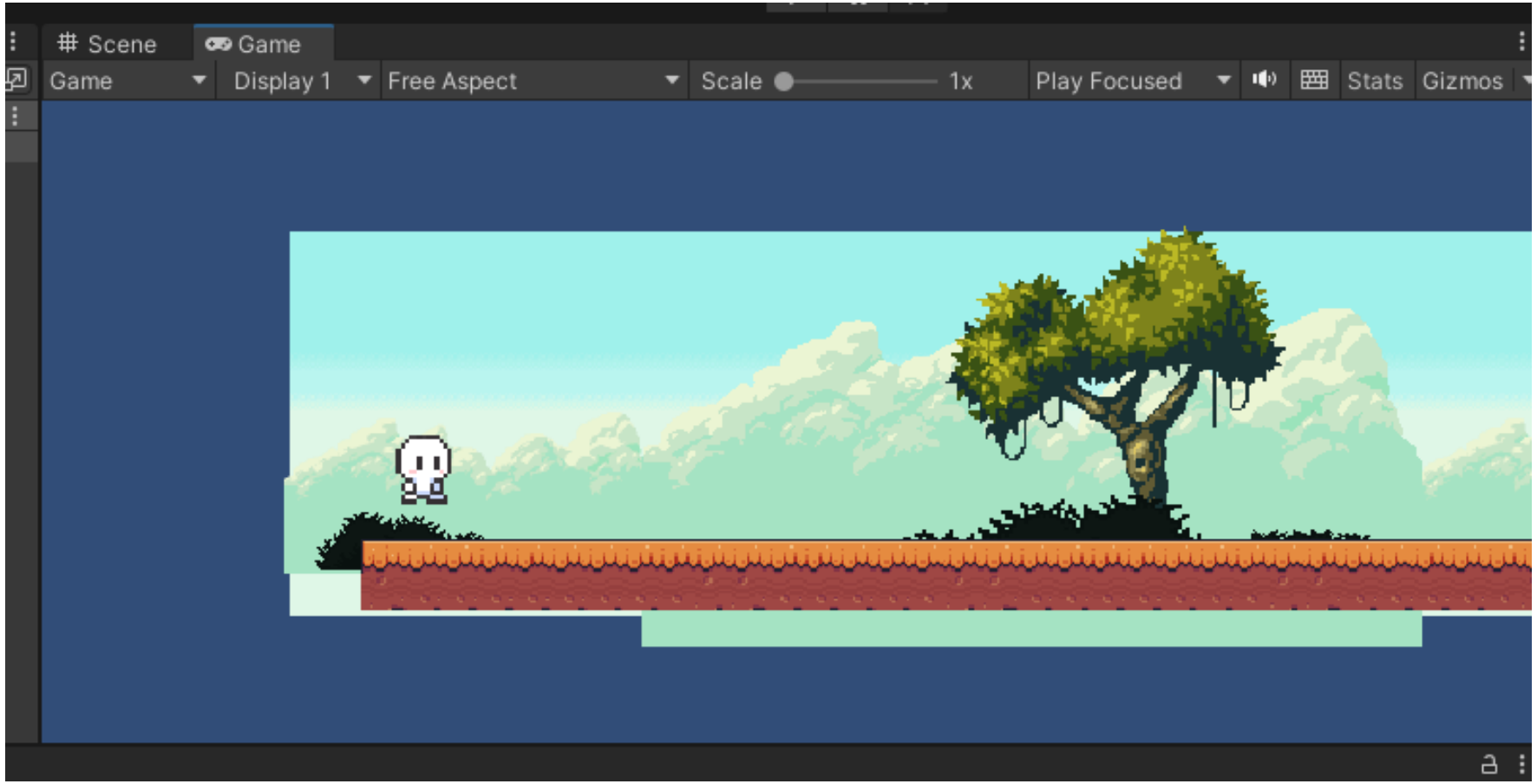
- In 2D games, the camera type is **Orthographic**, because we are working in 2 dimensions only.
- Since this is a 2D environment, make sure the Z value is **-10** by default.
- Change the Size setting to whatever's suitable.





Camera (Cont.)

- Size set to 7. Too big.



Camera (Cont.)

- Size set to 4. Just right.

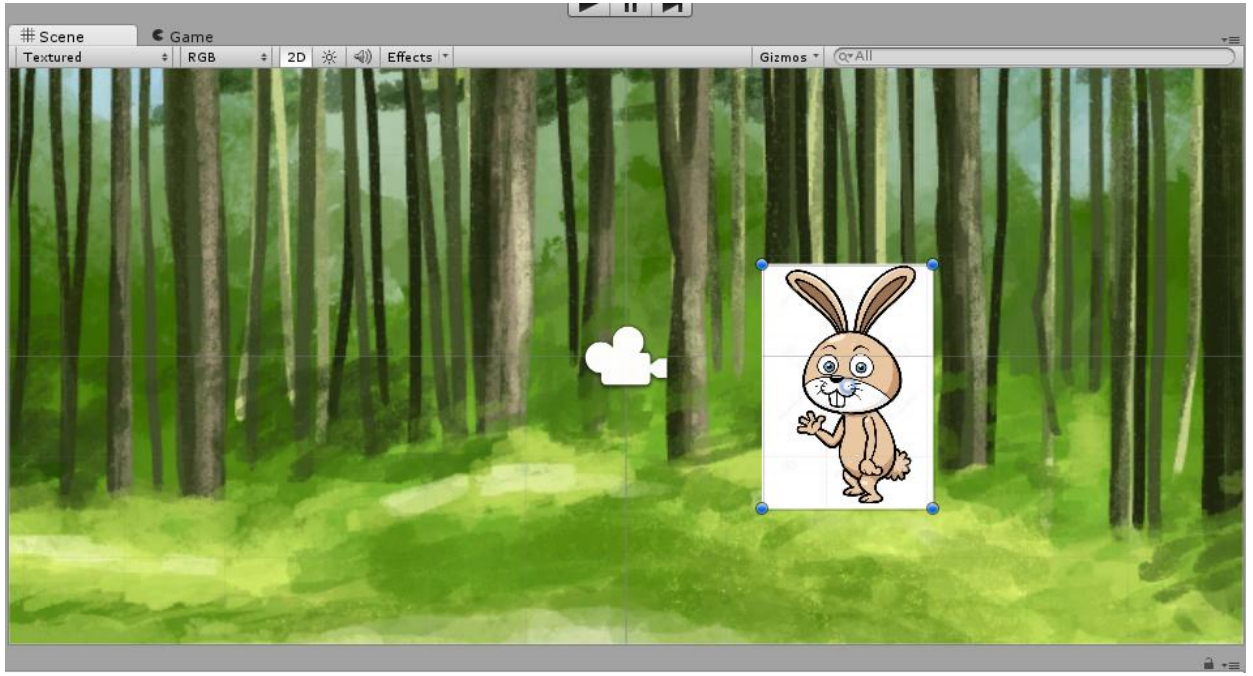


Quick Note

Never make changes to your scene while the Play button is clicked (Game Mode on), as they won’t be saved!

Types of Sprites

- In a 2D game, the art you use is called a Sprite.
- Your sprite must be **PNG** image filetype, to allow transparency
- If for example the sprite of your character does not have transparent a BG, you will have something like this:



Types of Sprites (Cont.)

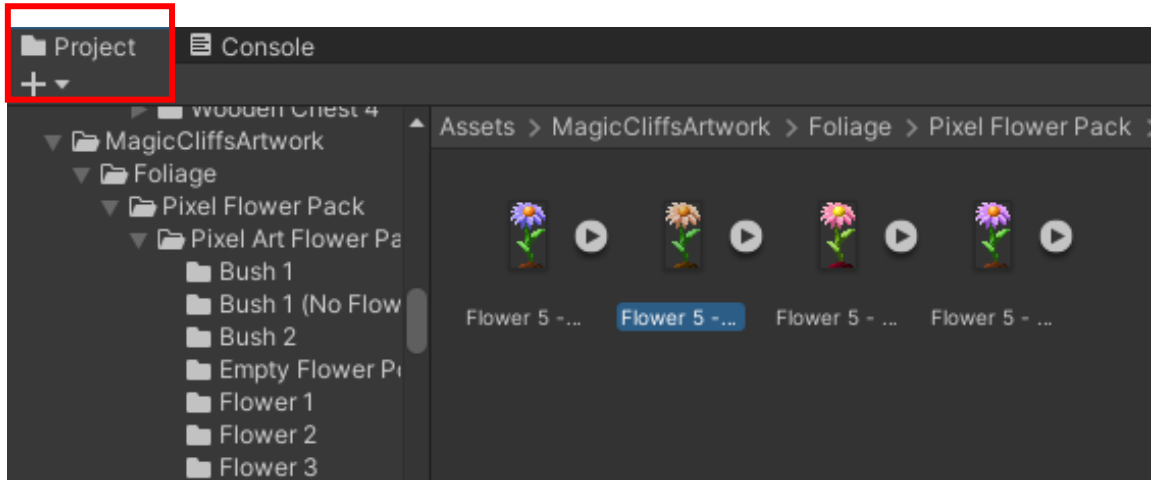
- But if your sprite has a transparent background, it will have gray and white checker marks as seen below:

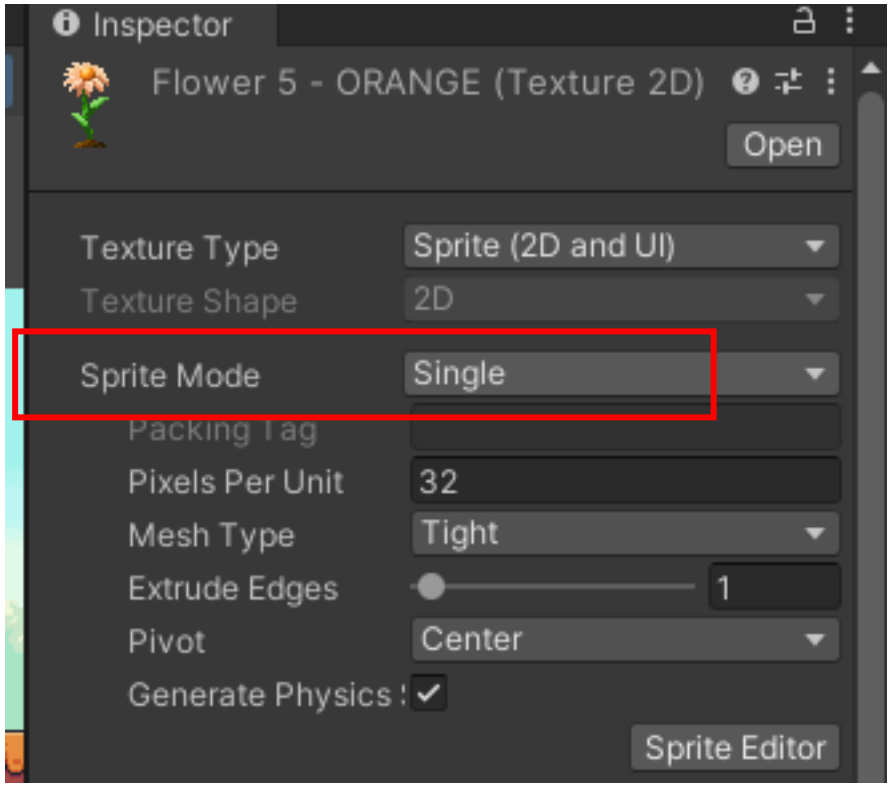


- Now *this* can be used in designing your level without problem

Types of Sprites (Cont.)

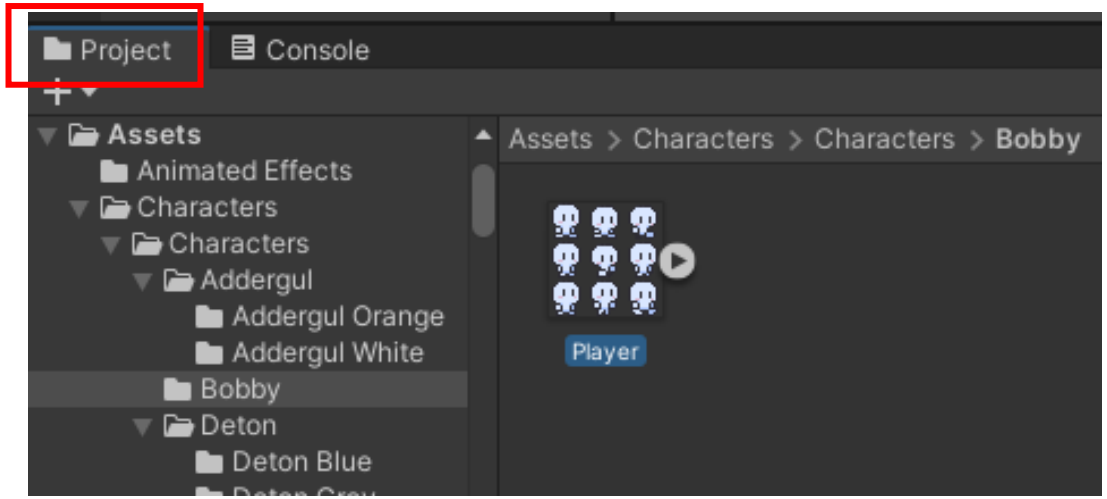
- A sprite in Unity can be 2 modes: single or multiple
- If a sprite is only 1 image, then make sure its mode is **single** while it is still in the Project window *before* you drag it onto the scene.

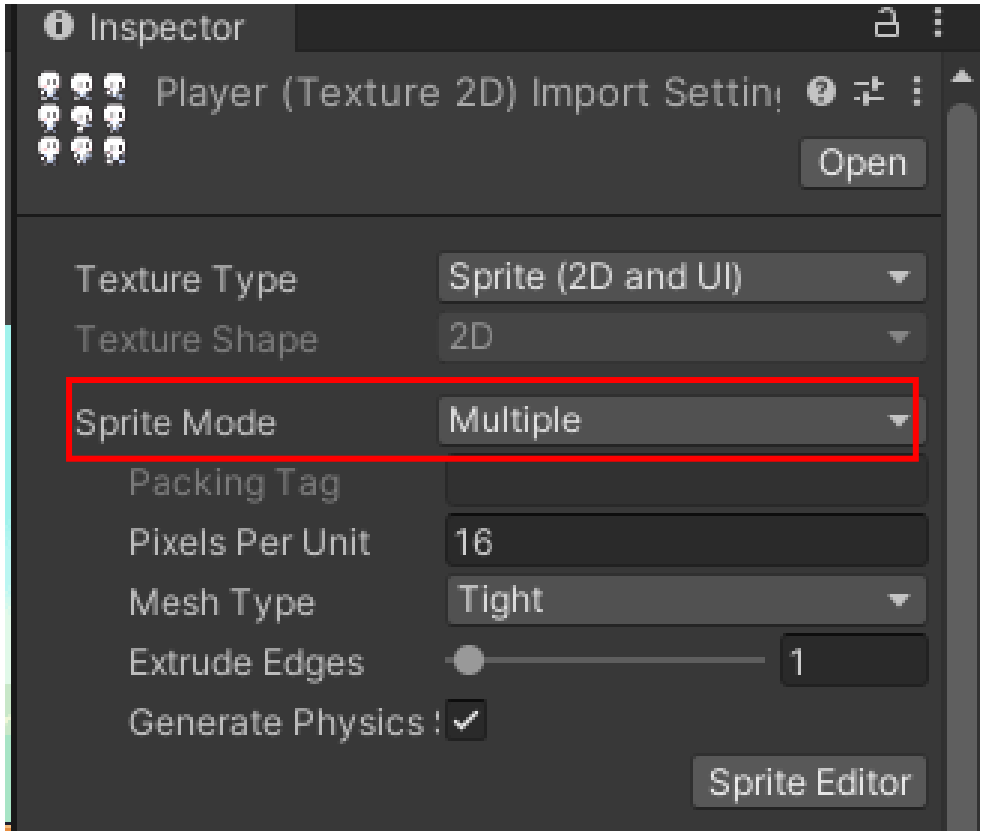




Types of Sprites (Cont.)

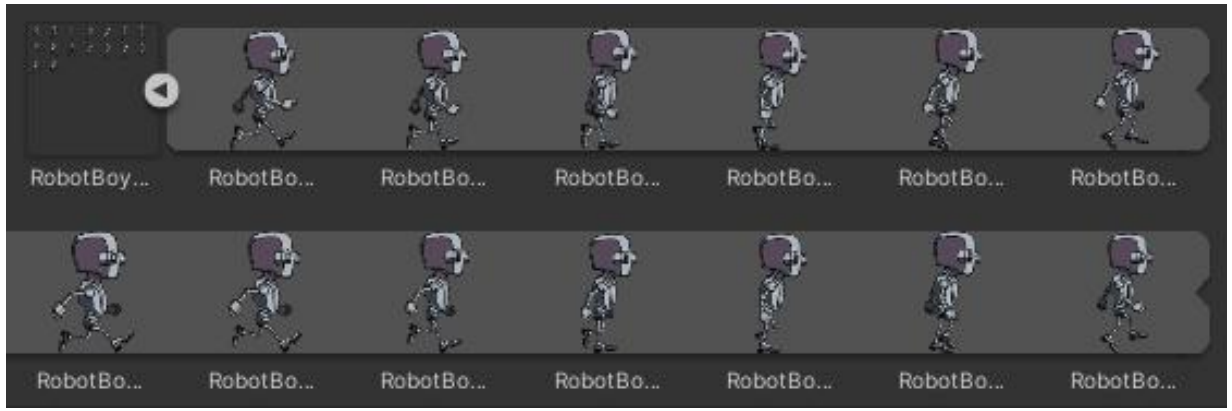
- If a sprite is several images, make sure its mode is ***multiple.***



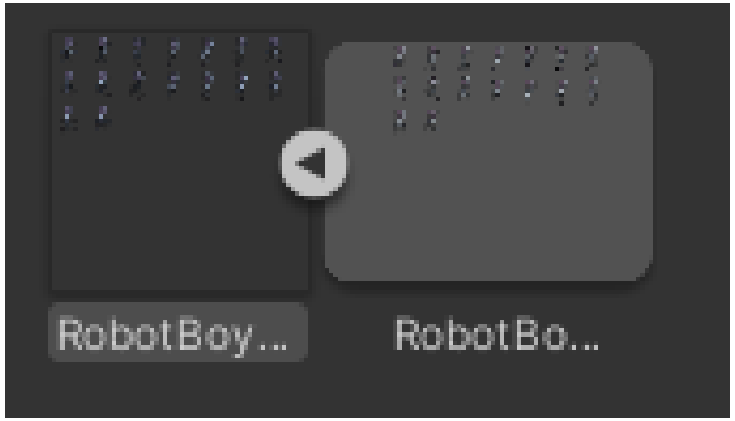


Sprite Editor

- In some cases, the sprite sheet is already cut up into individual images like this:

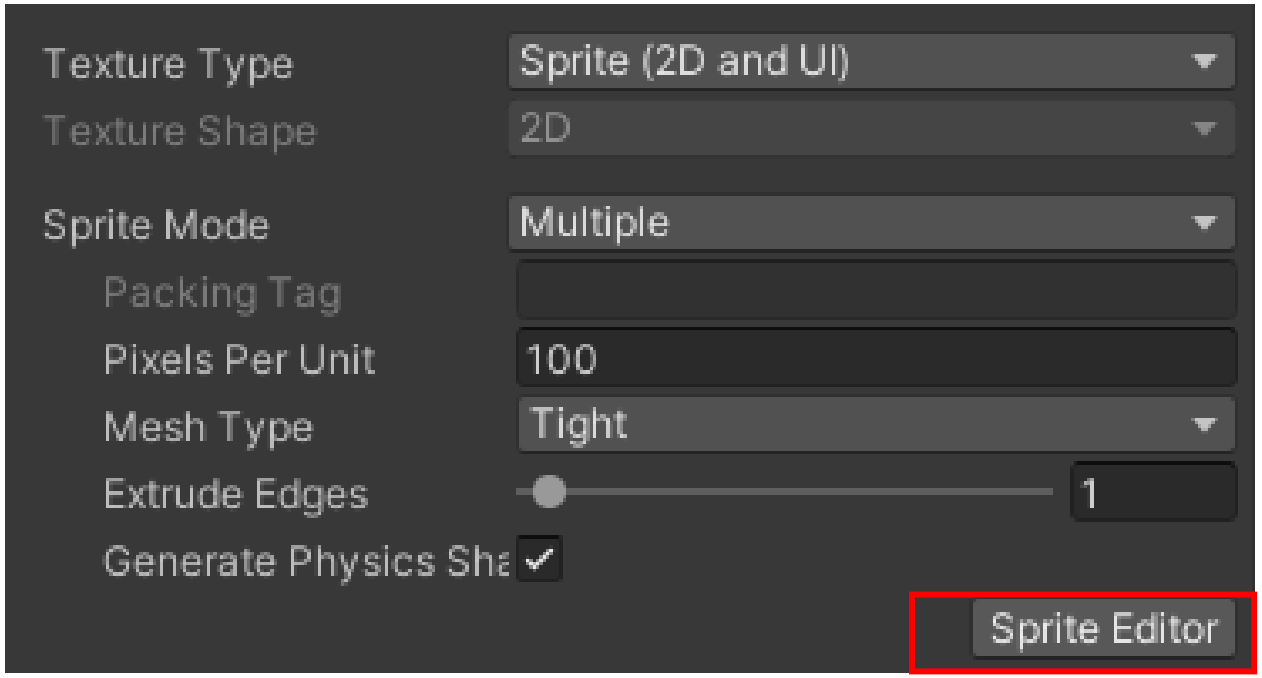


- But sometimes it's still one big image block like this:



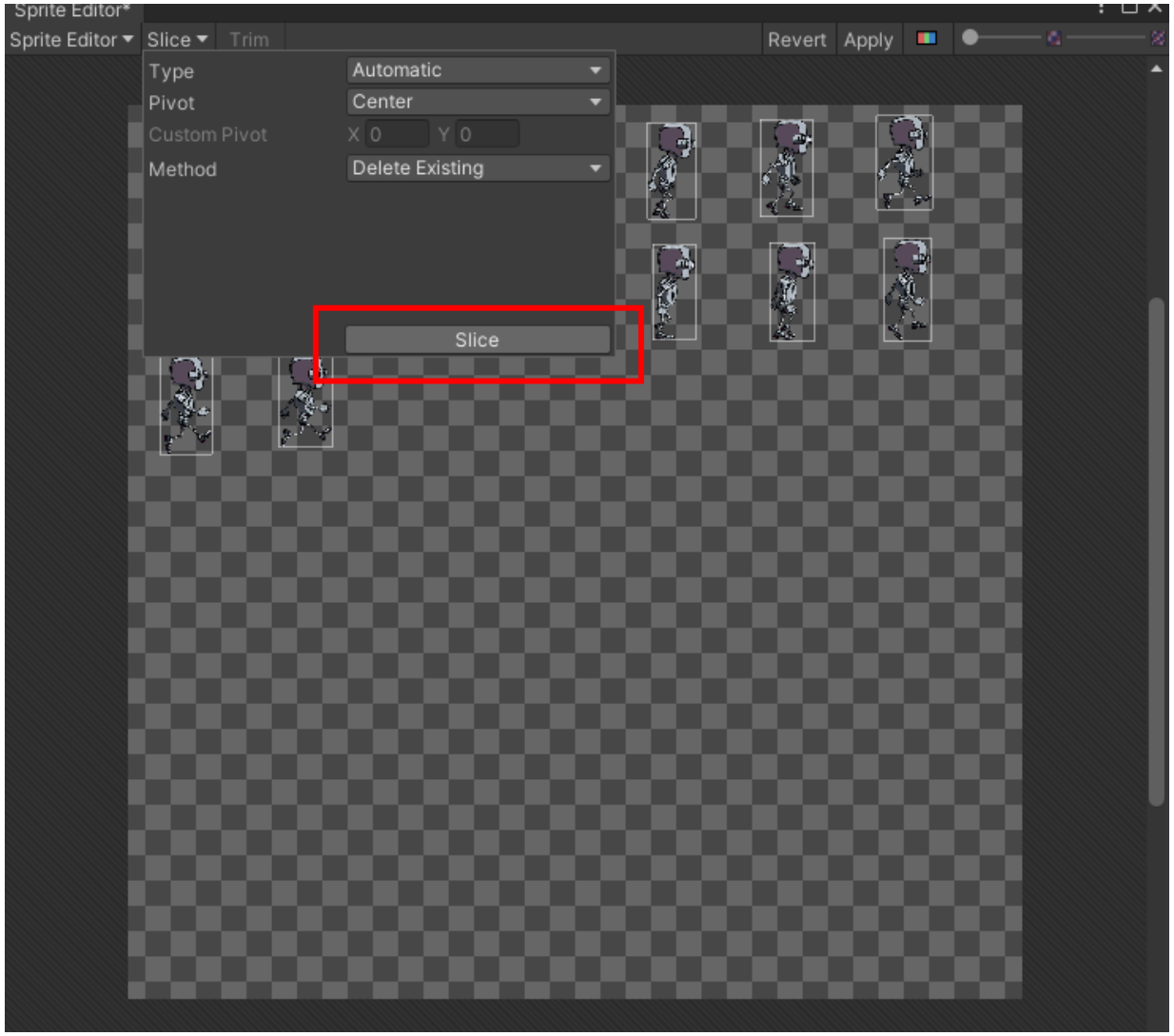
Exercise #04 – Slicing Sprites

- If you need to cut up the sprite sheet, you can do it inside Unity.
- Click the **Sprite Editor button** in the Inspector pane



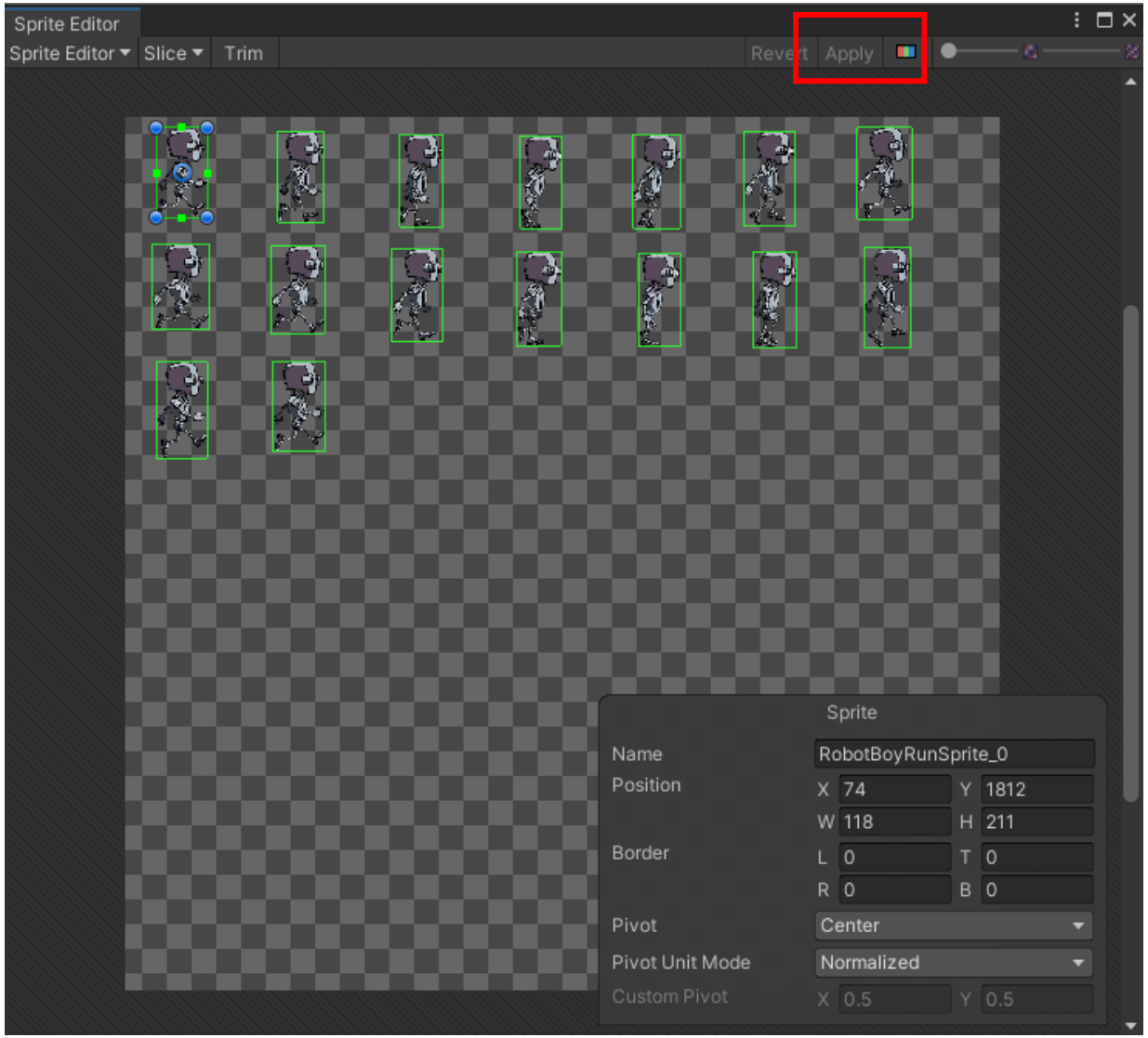
Exercise #04 – Slicing Sprites (Cont.)

- In the Editor, click the Slice button and Unity will automatically create outlines around each image in the sprite sheet.
- No need to change any other settings for now.



Exercise #04 – Slicing Sprites (Cont.)

- When you're done, click **Apply** on the right



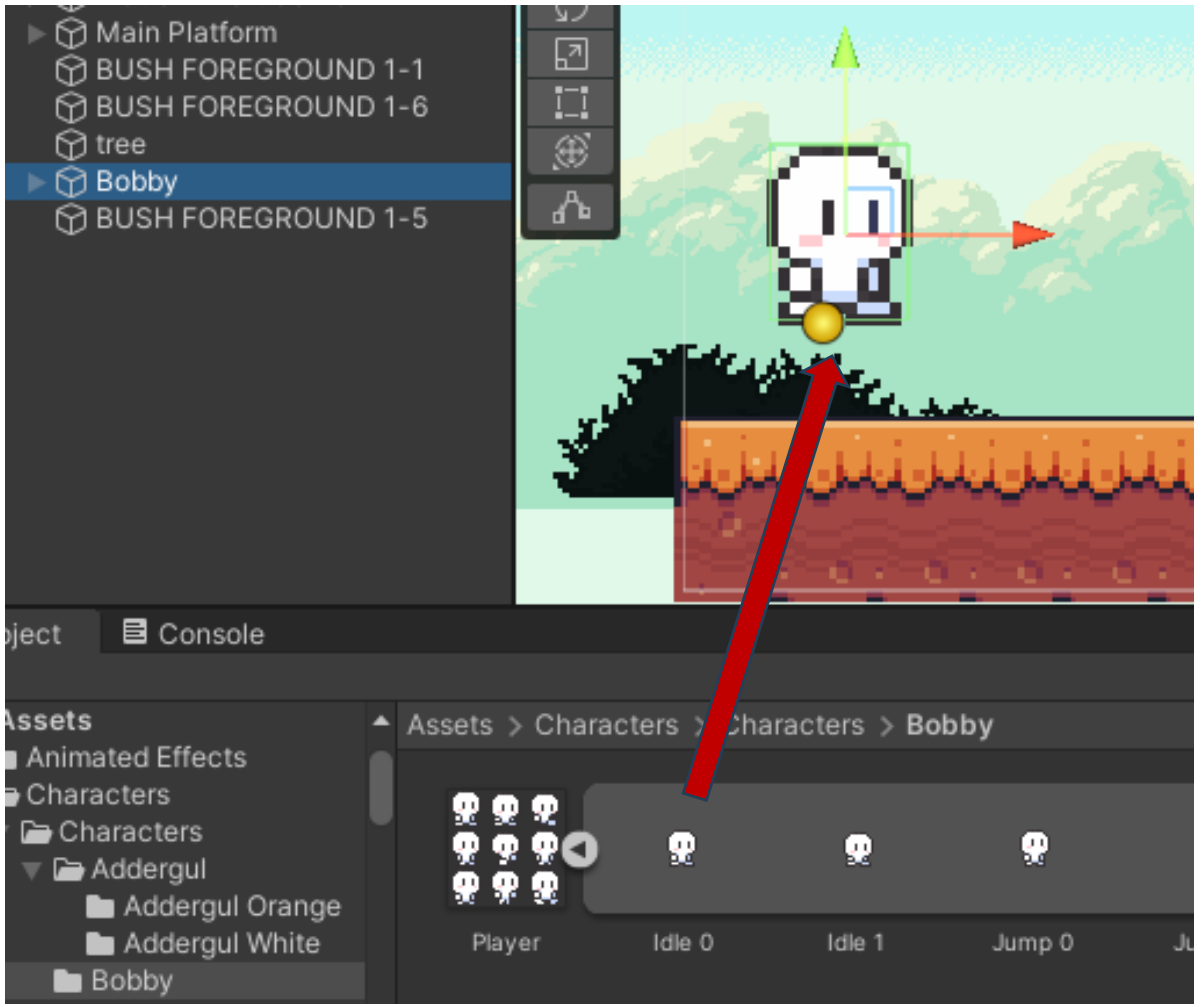
Exercise #05 - Create the Player GameObject

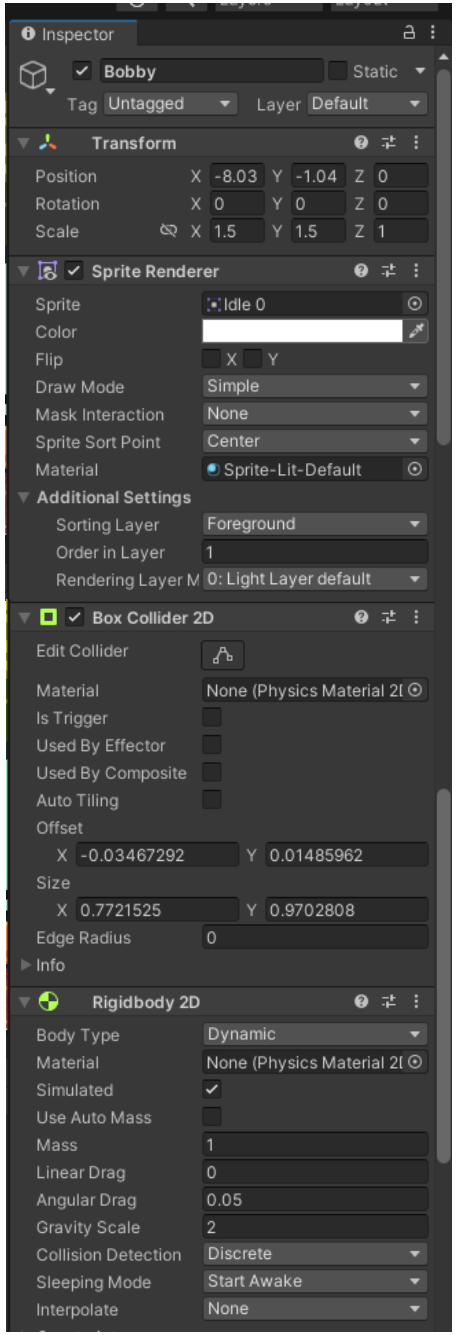
- Import a sprite of your character to the Project window in Unity

- Drag the character into the scene

- To make the character sprite solid and responsive to physics, don’t forget to add a **BoxCollider2D** and **Rigidbody2D**

Exercise #05 - Create the Player (Cont.)



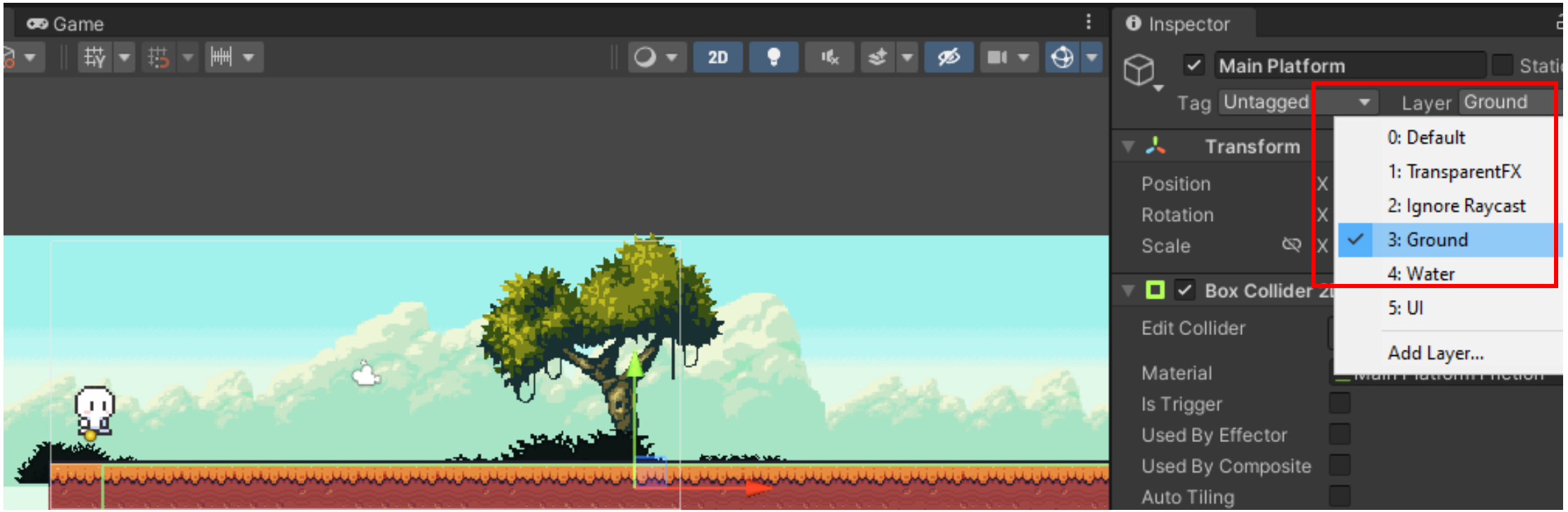


Exercise #05 - Create the Player (Cont.)

- In the RigidBody2D component:
  - Set the Interpolate property to Interpolate
    - (Interpolation can be useful to control jerky movement. With *interpolate* mode, motion is smoothed based on the object's positions in previous frames)
  - In the Constraints property, Freeze rotation around axis Z
- Create a BoxCollider2D
  - And make sure the box edges fit the character

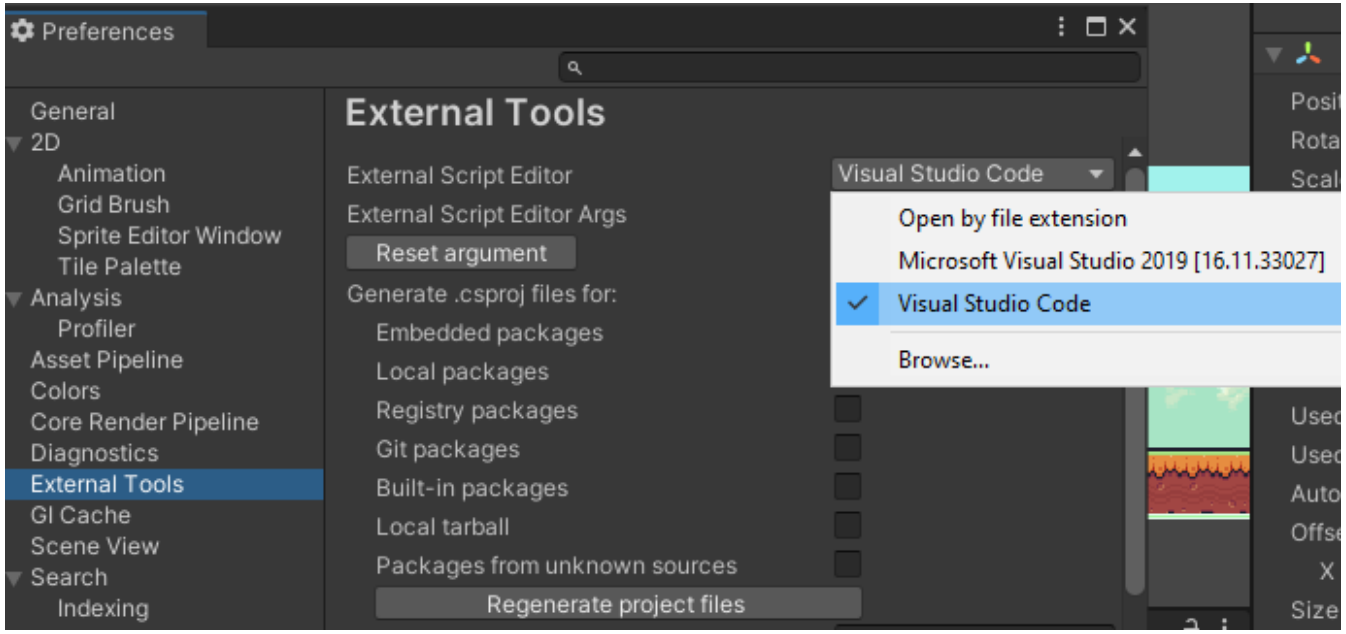
Exercise #05 - Create the Player (Cont.)

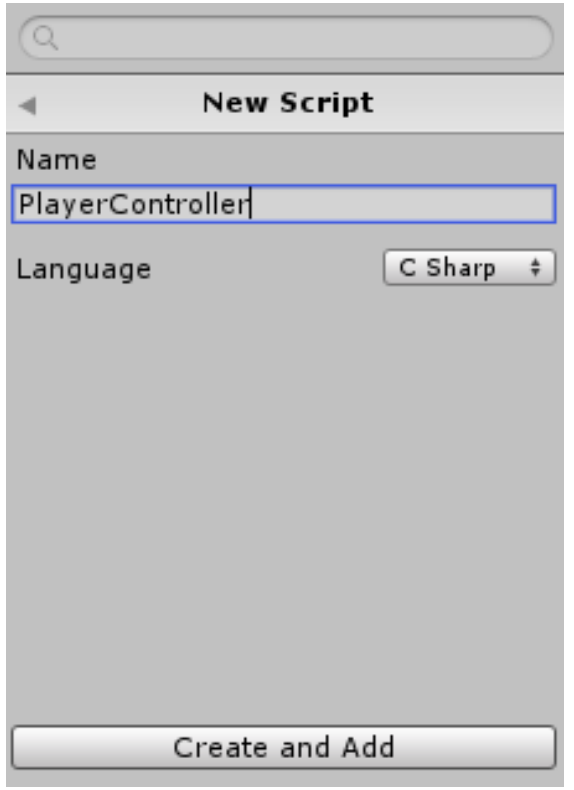
- Adjust the Ground Layer
  - Create a layer for solid ground that you want the character to stand on. Name it Ground

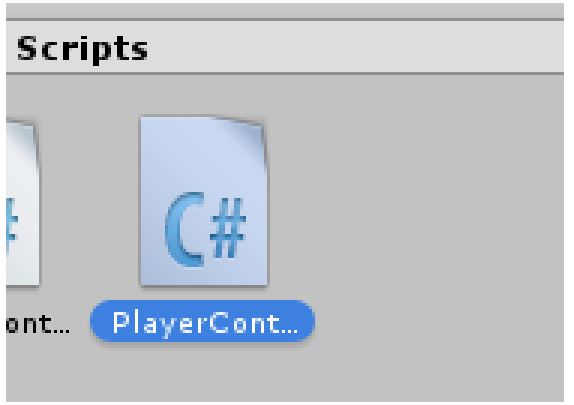


Exercise #06 - Adding a Script to Your Player

- To write code for the player, we must edit Unity's preference settings to use **Visual Studio code** as a script editor
- Go to Edit > Preferences > External Tools
- In the External Script Editor, choose VS Code from the list





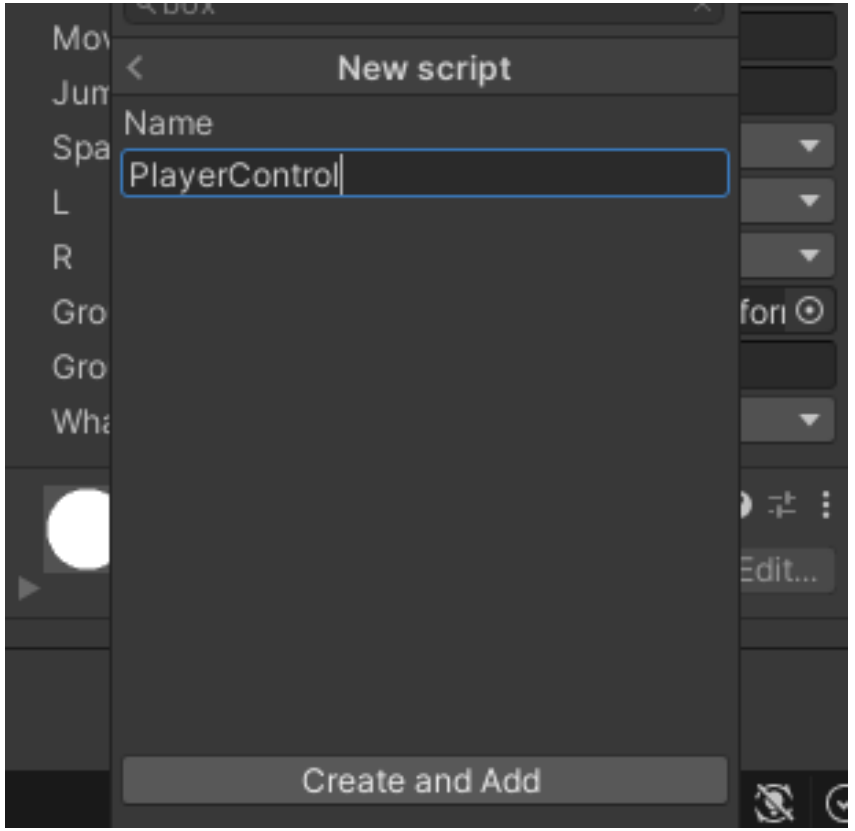


Exercise #06 - Adding a Script to Your Player (Cont.)

- Go back to your player, and click Add Component >New Script >C Sharp

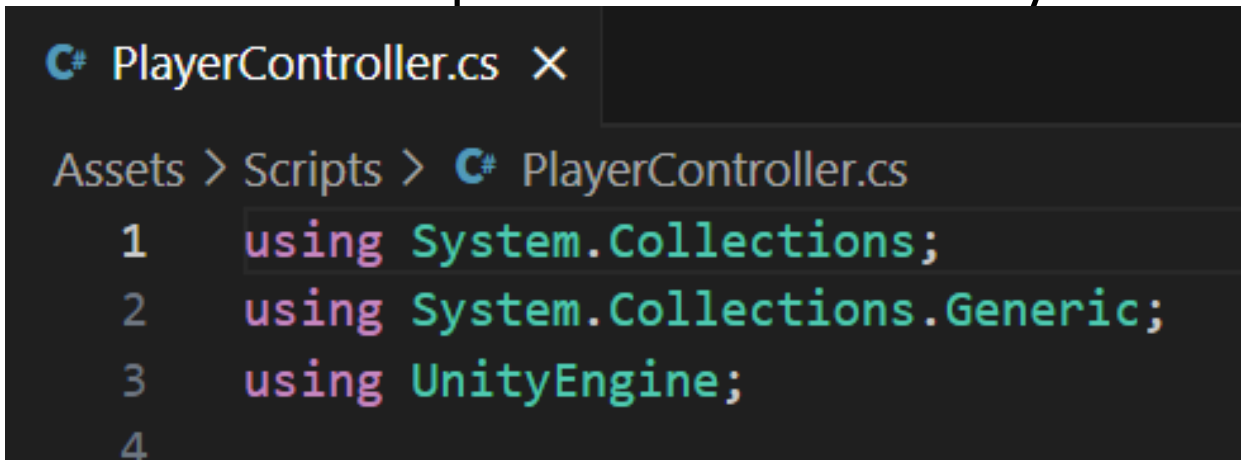
- Name the script PlayerController then Create and add.

- Double-click the script file in Project window to open it in VSCode.



Exercise #06 - Adding a Script to Your Player (Cont.)

- First, declare the essential namespaces we need in Unity:



- This script, like any script created for Unity, will inherit from **MonoBehaviour**.
  - *MonoBehaviour is a base class in Unity from which nearly all Unity scripts derive. It's the bridge between your C# code and Unity's game objects and engine functionalities.*

Exercise #06 - Adding a Script to Your Player (Cont.)

- Declare some variables:

```
public class PlayerController : MonoBehaviour {

    public float moveSpeed; //how fast the character moves
    public float jumpHeight; //how high the character jumps
    //private bool IsFacingRight; //check if player is facing right
    public KeyCode Spacebar; //Jump is the name we gave a keyboard button we chose to be the jump button.
    // In this case, we chose the Space button, and called it Spacebar. To allocate the Space button to the name
    // Spacebar, go to the Script
    //component of your player character, and choose Space from the dropdown list
    public KeyCode L;//L is the name we gave a keyboard button we chose to be the left movement button.
    public KeyCode R;//R is the name we gave a keyboard button we chose to be the right movement button.
```

Exercise #06 - Adding a Script to Your Player (Cont.)

- **The Start() function will be left empty for now.**

```
void Start () {
}
```

Exercise #06 - Adding a Script to Your Player (Cont.)

- The Update() function is called once every single frame in your game. This is where things like character movement and animation are added, because they are things that happen every frame the game is in play mode.

```
// Update is called once per frame
void Update () {

    if(Input.GetKeyDown(Spacebar)) //When user presses the space button ONCE
    {
        Jump(); //see function definition below
    }

    if (Input.GetKey(L)) //When user presses the left arrow button
    {
        GetComponent<Rigidbody2D>().velocity = new Vector2(-moveSpeed, GetComponent<Rigidbody2D>().velocity.y);
        //player character moves horizontally to the left along the x-axis without disrupting jump
    }

    if (Input.GetKey(R)) //When user presses the right arrow button
    {
        GetComponent<Rigidbody2D>().velocity = new Vector2(moveSpeed, GetComponent<Rigidbody2D>().velocity.y);
        //player character moves horizontally to the right along the x-axis without disrupting jump
    }
}

   void Jump()
   {
      GetComponent<Rigidbody2D>().velocity = new Vector2(GetComponent<Rigidbody2D>().velocity.x, jumpHeight);
      //player character jumps vertically along the y-axis without disrupting horizontal walk
   }
```

Exercise #06 - Adding a Script to Your Player (Cont.)

- To make your character sprite flip and face in the correct direction you want it to move, you have to access the SpriteRenderer component, then access the FlipX property. **True** will make the sprite face the right side, whilst **false** will flip it to face left.

- Update() function should be amended to have the addition you see on the next page:

Exercise #06 - Adding a Script to Your Player (Cont.)

```
if (Input.GetKey(L)) //When user presses the left arrow button
{
    GetComponent<Rigidbody2D>().velocity = new Vector2(-moveSpeed, GetComponent<Rigidbody2D>().velocity.y);
    //player character moves horizontally to the left along the x-axis without disrupting jump

    if(GetComponent<SpriteRenderer>()!=null)
    {
        GetComponent<SpriteRenderer>().flipX = true;
    }
}

if (Input.GetKey(R)) //When user presses the left arrow button
{
    GetComponent<Rigidbody2D>().velocity = new Vector2(moveSpeed, GetComponent<Rigidbody2D>().velocity.y);
    //player character moves horizontally to the right along the x-axis without disrupting jump

    if(GetComponent<SpriteRenderer>()!=null)
    {
        GetComponent<SpriteRenderer>().flipX = false;
    }
}
```

Exercise #06 - Adding a Script to Your Player (Cont.)

- Save your work and return to Unity. You'll find that the variables have been added as properties in the Script component in the Inspector window
- Give the variables values. You can use the screenshot as example



Exercise #06 - Adding a Script to Your Player (Cont.)

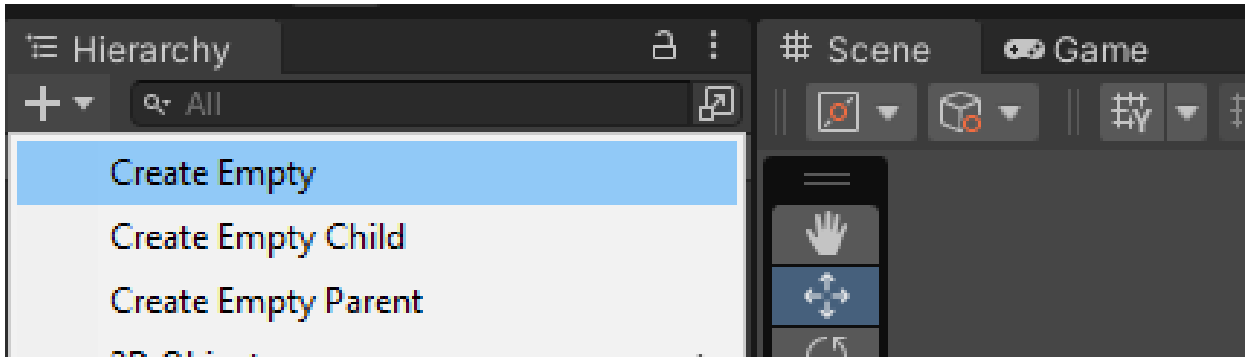
- Now go back to the script and add these variables:

```
public Transform groundCheck; //an invisible point in space. We use it to see if the player is touching
// the ground
public float groundCheckRadius; //a value to determine how big the circle around the player's feet is,
// and therefore determine how close he is to the ground
public LayerMask whatIsGround; //this variable stores what is considered a ground to the character
private bool grounded; //check if the character is standing on solid ground;
```

- We'll use these additions is to restrict the player's endless jumping and make it a **single jump,** meaning the player can't jump again until his feet is back on the ground.

Exercise #06 - Adding a Script to Your Player (Cont.)

- In the Hierarchy, create an Empty GameObject and name it groundCheck.



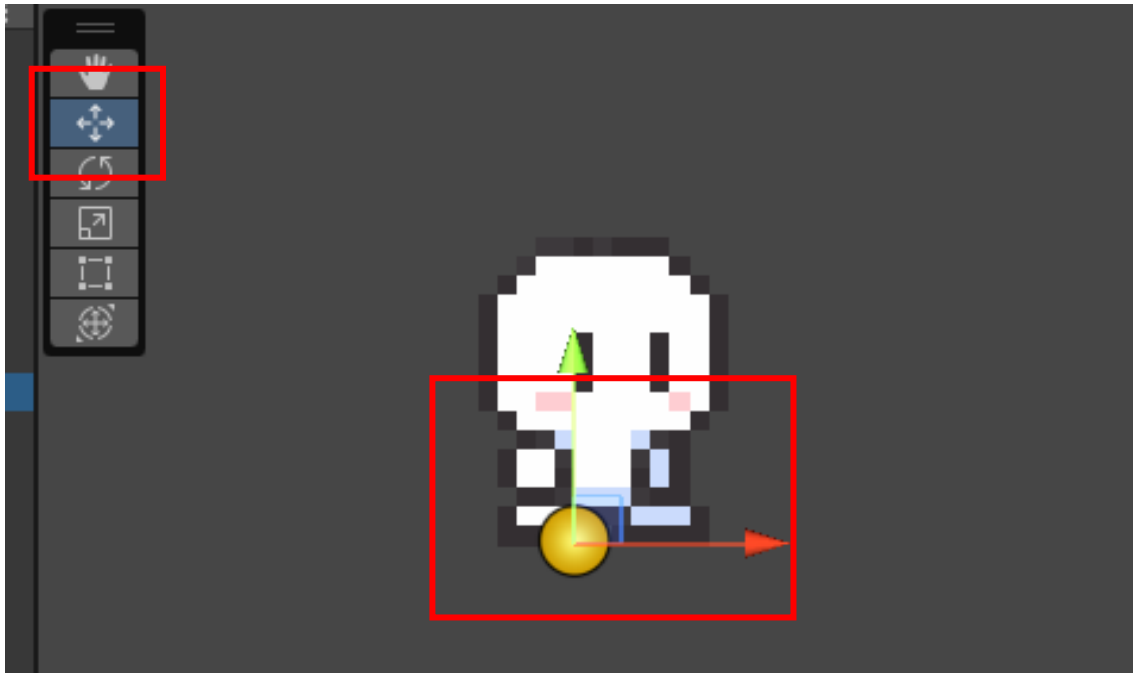
- Make it a *child* of Player GameObject by dragging it and dropping it on Player GameObject in the Hierarchy window.



Exercise #06 - Adding a Script to Your Player (Cont.)

- It is currently just an invisible point in the middle of the Scene window.

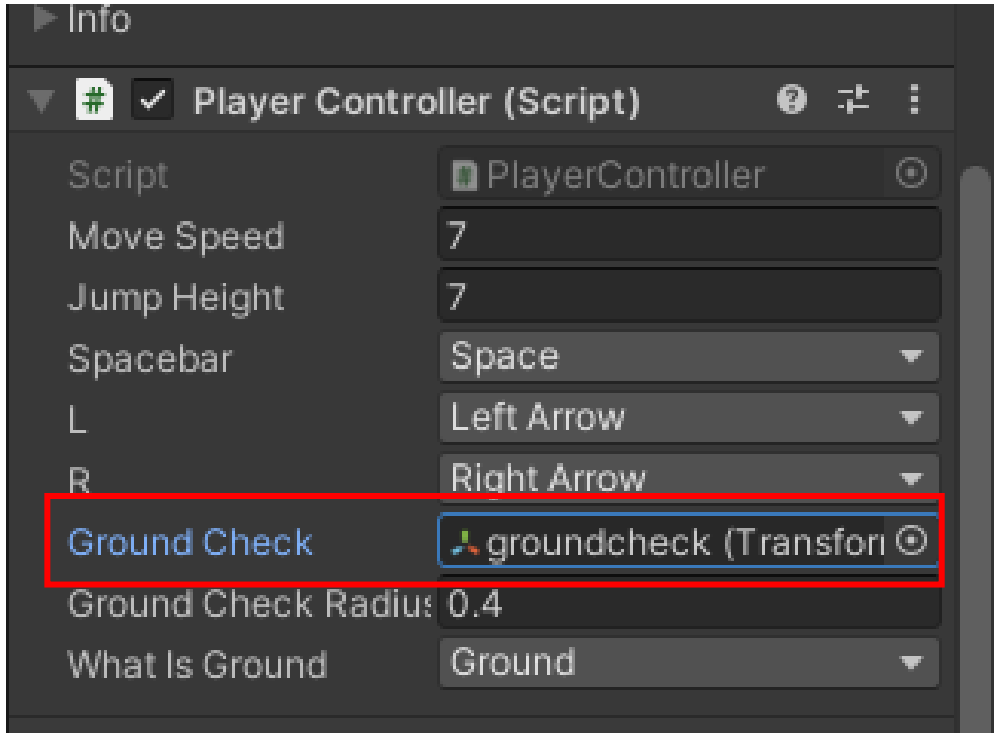
- Give it a color by going to its Transform component in the Inspector window. Choose a circle and use the Transform tool to position it at the character's feet.



Exercise #06 - Adding a Script to Your Player (Cont.)

- For the WhatIsGround property, select the Ground layer

- For the groundCheck variable, drag it from the Hierarchy window and drop it into the textbox in the Inspector window



Exercise #06 - Adding a Script to Your Player (Cont.)

- Create a FixedUpdate() function to calculate when the character is standing on ground

```
void FixedUpdate()
{
    grounded = Physics2D.OverlapCircle(groundCheck.position, groundCheckRadius, whatIsGround);
    //this statement calculates when exactly the character is considered by Unity's engine to be
    // standing on the ground.
}
```

- The groundCheck GameObject we have created is circular and has a radius. This statement calculates if the circle overlaps with the ground's boxcollider. If yes, the character is standing on the ground. If not, they are in the air.

- Note: Function names are case-sensitive! Make sure to write it correctly.

Exercise #06 - Adding a Script to Your Player (Cont.)

- Note on Update() vs FixedUpdate():
  - While the Update() function is called every single frame, FixedUpdate() is called every 0.2 sec by default. You can change that value later if your game needs it.
  - There is no need to check for the Grounded value every single frame, which is why it was placed in FixedUpdate().

Exercise #06 - Adding a Script to Your Player (Cont.)

- Now add **grounded** as a condition to restrict the jump to only once per click:

```
void Update () {

    if(Input.GetKeyDown(Spacebar) && grounded) //When user presses the space button ONCE
    {
        Jump(); //see function definition below
    }

    if (Input.GetKey(L)) //When user presses the left arrow button
    {
        GetComponent<Rigidbody2D>().velocity = new Vector2(-moveSpeed, GetComponent<Rigidbody2D>().velocity.y);
        //player character moves horizontally to the left along the x-axis without disrupting jump

        if(GetComponent<SpriteRenderer>()!=null)
        {
            GetComponent<SpriteRenderer>().flipX = true;
        }
    }

    if (Input.GetKey(R)) //When user presses the left arrow button
    {
        GetComponent<Rigidbody2D>().velocity = new Vector2(moveSpeed, GetComponent<Rigidbody2D>().velocity.y);
        //player character moves horizontally to the right along the x-axis without disrupting jump

        if(GetComponent<SpriteRenderer>()!=null)
        {
            GetComponent<SpriteRenderer>().flipX = false;
        }
    }
}
```

Useful References

- **Game Development Tutorials:**
  - **How to make a 2D Platformer – Unity Tutorial by Brackeys. URL retrieved from:**
  - https://www.youtube.com/playlist?list=PLPV2KyIb3jR42oVBU6K2DIL6Y22Ry9J1c
  - **Unity Tutorial - 2D Side Scroller (Super Platformer Bros) Episode 1. URL Retrieved from:** https://www.youtube.com/watch?v=BdlL5bwbCiI
  - **How to make a 2D RPG Game in Unity. URL retrieved from:** https://www.youtube.com/playlist?list=PL_4rJ_acBNMH3SExL3yIOzaqj5IP5CJLC
  - **Unity Tutorial - 2D RPG Tutorial Playlist. URL Retrieved from:** https://www.youtube.com/playlist?list=PL_S_hCof18KzyQ2sl4AfjSvmo1ILCuywH
  - **Unity 2D Tile Mapper. URL retrieved from:** https://www.youtube.com/watch?v=_x0bMTxP7Yw

Useful References

- Johnson, M., Hasankolli, R., & Henley, J. A. (2014). *Learning 2D Game Development with Unity: A Hands-on Guide to Game Creation*. Pearson Education.
- **2D Character Controllers. URL retrieved from:** https://unity3d.com/learn/tutorials/modules/beginner/2d/2d-controllers?playlist=17093
- **Unity's New Input System in just 9 Minutes. URL retrieved from:** https://www.youtube.com/watch?v=ptDoB00P8QY&list=PLJBcv4t1EiSxk9UuMkdk_14GBvEVQgsIW&index=3
- **How to make a 2D Platformer – Unity Tutorial by Brackeys. URL retrieved from:** https://www.youtube.com/playlist?list=PLPV2KyIb3jR42oVBU6K2DIL6Y22Ry9J1c
- **Unity 2D Platformer Tutorial - Part 1 - Moving and Jumping. URL retrieved from:** https://www.youtube.com/watch?v=86Bgt--Ww7w
- **The Difference Between Update() and FixedUpdate. URL retrieved from:** https://unity3d.com/learn/tutorials/topics/scripting/update-and-fixedupdate